

Non-Uniform Time Grid, Take 2: Defining the x - and y -Variable Grids

Optimal Transport / Schrödinger Bridge Project

Abstract

This is a step-by-step, checkpointed re-derivation of the non-uniform (Chebyshev-clustered) time grid used by the Python LADMM solver, written to be verified one section at a time rather than accepted in one pass. **Step 1 (this section only): define the two grids the splitting actually carries state on – the x -variable (constraint) grid and the y -variable (KE) grid – and lay out where every field lives for the concrete case $n_t = n_x = 4$, space still uniform (1D).** No operators, no FP constraint, no projection yet – those are later steps, once the grid itself is confirmed correct.

1 Naming: what “ x ” and “ y ” mean here

This is a re-statement of an assumption, flagged for you to confirm or correct before anything below is built on it. The solver (`ladmm.py`) solves a consensus-form problem

$$\min_{x,y} f_1(x) + f_2(y) \quad \text{s.t.} \quad Ax + By = d,$$

where x and y are *not* spatial coordinates – they are the two copies of the state that linearized ADMM carries:

- $x = (q, b)$, code fields `rho_stag/mx_stag`: lives on the **constraint grid**, staggered, where the Fokker–Planck constraint is enforced *exactly* (`projection_nu.py`).
- $y = (\rho, m)$, code fields `rho_cc/mx_cc`: lives on the **KE grid**, collocated, where the kinetic-energy prox acts pointwise (`prox_nu.py`).

As ambient (real) vector spaces – shapes derived in §4, from the staggered/collocated layouts of `operators.py/operators_nu.py` –

$$x = (q, b) \in \mathbb{R}^{(n_t-1)n_x} \times \mathbb{R}^{n_t(n_x-1)}, \quad y = (\rho, m) \in \mathbb{R}^{n_t n_x} \times \mathbb{R}^{n_t n_x}. \quad (1)$$

d is the ADMM constant, **not** `ladmm.py`’s own `b` parameter (that name is unavailable here regardless – b already names the x -variable’s momentum field) – and, contrary to an earlier version of this note, **not** generally zero either. `pipeline_nu.py` does zero out its own `b`, but only because its `A_fn` is itself affine (bakes ρ_0, ρ_1 directly into the interpolation via `interp_t_at_phi`’s boundary arguments) rather than exposing that boundary part separately; once A is refactored to be genuinely linear (Step 5, §10), the leftover boundary contribution becomes exactly this d , generally nonzero. This matches `nonuniform_grid_refine_ends.tex` §2’s “two grids.” *Space* (the physical coordinate the density lives over) is a separate axis from either grid – right now it is 1D and uniform; both grids share it. **If “ x and y ” was meant as two spatial dimensions instead, stop here and say so – everything past this point assumes the ADMM-splitting reading above.**

2 The time grid: Chebyshev–Gauss–Lobatto clustering

n_t cells on $[0, 1]$, edges $t_0 = 0 < t_1 < \dots < t_{n_t} = 1$. The reference (uniform) partition is $\xi_j = j/n_t$, $j = 0, \dots, n_t$. The clustered edges are the Chebyshev–Gauss–Lobatto map applied to ξ :

$$t_j = \frac{1}{2}(1 - \cos(\pi \xi_j)), \quad j = 0, \dots, n_t. \quad (2)$$

(Code: `clustered_time_grid(nt, strength=1.0)` in `time_grid.py` – taking `strength=1` recovers exactly (2); `strength < 1` blends toward the uniform $t_j = \xi_j$ and is a separate knob, not used below.) Widths and centers follow the grid’s own general construction (`_time_grid_from_edges`):

$$\Delta t_j = t_{j+1} - t_j \quad (j = 0, \dots, n_t - 1), \quad \tau_j = \frac{1}{2}(t_j + t_{j+1}).$$

Clustering is quadratic at both ends: near $\xi = 0$, $t \approx \frac{\pi^2}{4}\xi^2$, so the first cell has width $O(1/n_t^2)$ instead of the uniform grid’s $O(1/n_t)$; by symmetry ($t_j = 1 - t_{n_t-j}$, since $\cos(\pi(1 - \xi)) = -\cos(\pi\xi)$) the same holds at $\xi = 1$. Mid-domain the map is close to linear, so cell widths there are close to uniform.

3 The space grid: uniform (1D, for now)

Unchanged from `grid_nu.py/grid.py`: n_x cells on $[0, L]$ ($L = 1$ below), uniform width $\Delta x = L/n_x$, edges $x_k = k \Delta x$, cell centers $\xi_k = (k - \frac{1}{2})\Delta x$.¹ No clustering here yet – this section is a placeholder for whatever the analogous space construction turns out to be, once the time grid above is confirmed.

4 Where each field lives

Combining §2–3 with the staggered/collocated layouts from `operators.py/operators_nu.py` (docstring: “rho lives at $(n_t - 1, n_x)$ – time-interior nodes, space cell-centers; mx lives at $(n_t, n_x - 1)$ – time cell-centers, space-interior nodes”):

variable	field	locations	shape (n_t, n_x general)	shape ($n_t=n_x=4$)
$x = (q, b)$	q (density)	$(t_j, \xi_k), j = 1, \dots, n_t - 1$	$(n_t - 1) \times n_x$	$3 \times 4 = 12$
	b (momentum)	$(\tau_i, x_k), k = 1, \dots, n_x - 1$	$n_t \times (n_x - 1)$	$4 \times 3 = 12$
$y = (\rho, m)$	ρ (density)	(τ_i, ξ_k) , both centers	$n_t \times n_x$	$4 \times 4 = 16$
	m (momentum)	(τ_i, ξ_k) , both centers	$n_t \times n_x$	$4 \times 4 = 16$

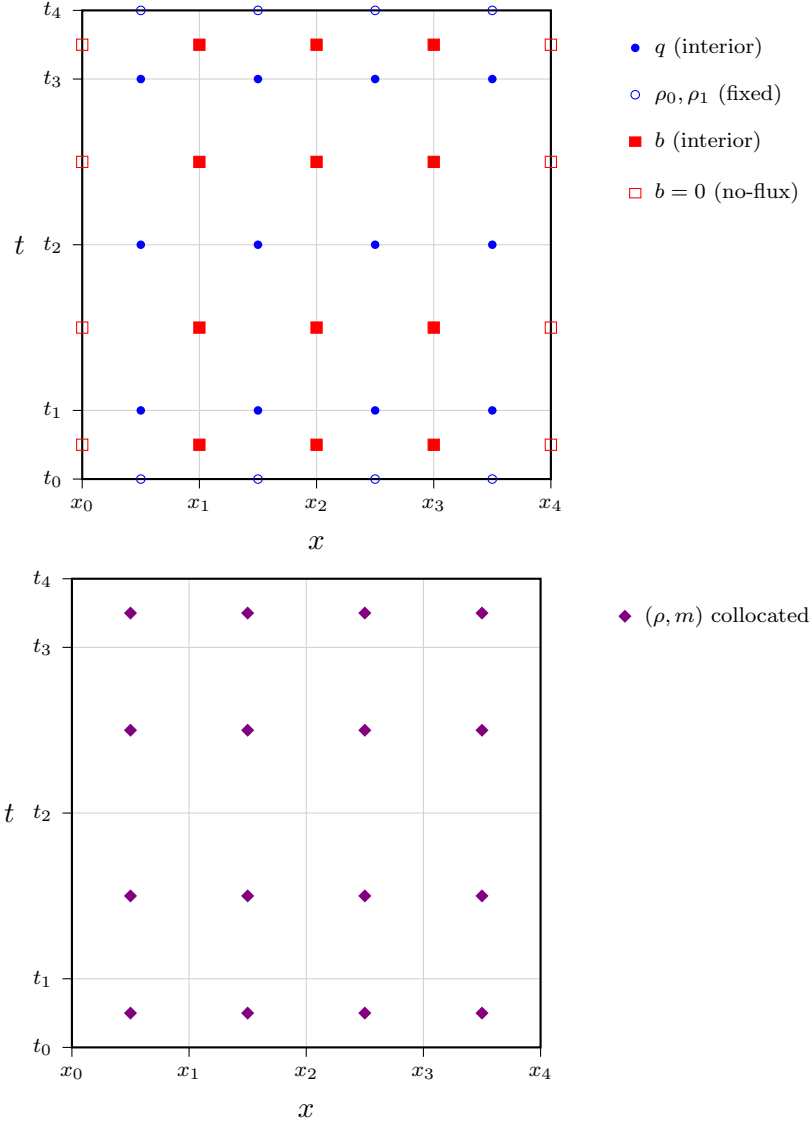
Flattening each field’s 2D array of samples into a single real vector, in either order, is exactly §1’s (1): $q \in \mathbb{R}^{(n_t-1)n_x}$, $b \in \mathbb{R}^{n_t(n_x-1)}$, $\rho, m \in \mathbb{R}^{n_t n_x}$.

Boundary data (fixed, not part of the unknowns): $q(t_0, \cdot) = \rho_0$, $q(t_{n_t}, \cdot) = \rho_1$ (the two marginals, sampled on the space grid of §3); $b(\cdot, x_0) = b(\cdot, x_{n_x}) = 0$ (no-flux). Every t_j , τ_i above is the *clustered* value from §2 – this is the only thing that changes relative to the uniform-time solver; every space location is unchanged from §3.

¹Using ξ_k for the space cell centers here follows `nonuniform_grid_refine_ends.tex`’s notation; unrelated to ξ_j in §2, which is the time reference partition. Sorry for the clash – kept for consistency with the earlier doc.

5 Schematic: $n_t = n_x = 4$

Horizontal axis $x \in [0, 1]$ (uniform); vertical axis $t \in [0, 1]$ (clustered, drawn to true scale – the bunching near both ends is the actual geometry, not exaggerated). Left: x -variable (constraint grid), staggered – q (circles) at time-edges $\times$ space-centers, b (squares) at time-centers $\times$ space-edges, boundary data (hollow) at the domain edges. Right: y -variable (KE grid), collocated – (ρ, m) (diamonds) at time-centers $\times$ space-centers only.



Note the visible clustering: the topmost/bottommost rows of $q/(\rho, m)$ points (and the outer two rows of b -centers τ_0, τ_3) sit noticeably closer to $t = 0/t = 1$ than the two middle rows, since §2's $\Delta t_0 = \Delta t_3 \approx 0.146 < \Delta t_1 = \Delta t_2 \approx 0.354$. Space spacing is uniform throughout (equal column spacing in both panels).

6 Step 1: confirmed

1. **Naming** (§1): $x = (q, b)$ on the constraint grid, $y = (\rho, m)$ on the KE grid. Confirmed.
2. **Clustering**: full Chebyshev–Gauss–Lobatto (§2, `strength=1.0`), not the blended family. Confirmed.
3. **Space**: uniform for this step (§3 stays a placeholder); space-side clustering is out of scope until the time grid is settled. Confirmed.
4. **Field placements** (§4, §5): correct as drawn. Confirmed – so whatever “the wrong way” referred to is *not* the grid layout itself, and must be in something this step deliberately hasn’t touched yet: the discrete FP constraint on the clustered grid, the projection that enforces it, or the KE prox’s per-row weighting. Step 2 should pick one of those.

7 Step 2: the kinetic-energy functional f_2

Discretize-first, optimize-second: write down the continuous objective, then discretize it directly by quadrature on the y -grid already fixed in Step 1 – no PDE, no optimality conditions, no prox yet.

7.1 The continuous functional

The kinetic energy of a curve of measures (ρ, m) on $[0, 1] \times [0, L]$ is

$$\mathcal{F}_2[\rho, m] = \int_0^1 \int_0^L L(\rho(t, x), m(t, x)) \, dx \, dt, \quad (3)$$

where $L : \mathbb{R} \times \mathbb{R} \rightarrow (-\infty, +\infty]$ is the lower-semicontinuous closure of $(\rho, m) \mapsto m^2/(2\rho)$:

$$L(\rho, m) = \begin{cases} \frac{m^2}{2\rho}, & \rho > 0, \\ 0, & \rho = 0, \, m = 0, \\ +\infty, & \text{otherwise } (\rho = 0, \, m \neq 0 \text{ or } \rho < 0). \end{cases} \quad (4)$$

L is the perspective function of $\phi(u) = u^2/2$ (i.e. $L(\rho, m) = \rho \phi(m/\rho)$ for $\rho > 0$), closed by taking its lsc envelope at $\rho = 0$: since ϕ is convex with superlinear growth, the only direction along which $L(\rho, m)$ stays finite as $\rho \rightarrow 0^+$ is $m \rightarrow 0$ at the matching rate $m = O(\rho)$ (any fixed $m \neq 0$ blows up like $1/\rho$). This closure is what makes L jointly convex and lsc on all of $\mathbb{R} \times \mathbb{R}$ (not just $\{\rho > 0\}$) – the standard construction for the Benamou–Brenier kinetic energy, needed here because the optimal ρ can genuinely vanish on part of the domain (e.g. Schrödinger-bridge solutions can be zero off the support of ρ_0, ρ_1 ’s convex hull). `prox_ke_cc`’s `neg` clamp (`rho_new, mx_new` $\rightarrow 0$ whenever the cubic solve returns $\rho_{\text{new}} \leq 10^{-12}$) is exactly imposing this closure numerically: the only finite point on $\{\rho = 0\}$ is the origin, so the prox may only land *there*, never at $\rho = 0$ with $m \neq 0$.

7.2 Discretize: Riemann sum on the y -grid

Step 1 fixed the y -grid at (τ_i, ξ_k) , $i = 0, \dots, n_t - 1$, $k = 1, \dots, n_x$, each point the exact center of a cell of size $\Delta t_i \times \Delta x$ (midpoint rule, exact for the cell-average of a linear function, same order of accuracy on the clustered grid as on the uniform one – Δt_i varies by row but each cell’s own midpoint is still used for its own quadrature). The direct quadrature discretization of (3) on this grid is

$$f_2(\rho, m) = \sum_{i=0}^{n_t-1} \sum_{k=1}^{n_x} \Delta t_i \Delta x L(\rho_{i,k}, m_{i,k}), \quad \rho_{i,k} := \rho(\tau_i, \xi_k), \quad m_{i,k} := m(\tau_i, \xi_k), \quad (5)$$

with Δt_i the *clustered* value from Step 1 §2 and $\Delta x = L/n_x$ constant (Step 1 §3). On the uniform-time grid ($\Delta t_i \equiv \Delta t = 1/n_t$ for all i), (5) collapses to $f_2 = \Delta t \Delta x \sum_{i,k} L(\rho_{i,k}, m_{i,k})$ – a single global constant times an unweighted sum.

To verify before Step 3

1. Is (3)–(4) the functional you meant – kinetic energy only, no ε (diffusion belongs to the FP constraint, i.e. f_1 /the projection, not here)?
2. Is the discretize-first quadrature (5) – midpoint rule, cell $\Delta t_i \times \Delta x$ centered at (τ_i, ξ_k) – the intended discretization, or did you have a different quadrature in mind (e.g. one that doesn’t use the cell *center* as the sample point)?

Not doing yet: reconciling (5) against `prox_ke_cc/ prox_ke_cc_nu`’s actual code – dropped per your flag that something about that formulation is off. Once §7 itself is settled, we can redo that comparison from scratch rather than patch the previous version.

8 Step 3: the Fokker–Planck constraint

8.1 Continuous constraint

$$\partial_t \rho + \partial_x m = \varepsilon \partial_x^2 \rho, \quad (t, x) \in (0, 1) \times (0, L), \quad (6)$$

with $\rho(0, \cdot) = \rho_0$, $\rho(1, \cdot) = \rho_1$ (the two fixed marginals) and, at both space boundaries, no advective *and* no diffusive flux: $m(t, 0) = m(t, L) = 0$ and $\partial_x \rho(t, 0) = \partial_x \rho(t, L) = 0$ (fully reflecting domain).

8.2 Where it lands

Discretize-first: $x = (q, b)$ carries the samples, but the residual is evaluated – landed – on the *same* grid as y fixed in Step 1/2, (τ_i, ξ_k) , $i = 0, \dots, n_t - 1$, $k = 1, \dots, n_x$. So the residual $R(q, b)$ is a field in $\mathbb{R}^{n_t \times n_x}$ (flattened, $\mathbb{R}^{n_t n_x}$) – the same ambient space as *each single field* of y in (1), not the same space as x itself.

8.3 The matrices

Four building blocks, each acting along one axis only.

Time derivative $\mathbf{D}_t$ (acts on each space column $q_{\cdot,k} \in \mathbb{R}^{n_t-1}$ independently, using Step 1's clustered Δt_i):

$$(D_t q)_{i,k} = \frac{q_{i+1,k} - q_{i,k}}{\Delta t_i}, \quad q_{0,k} = \rho_0(\xi_k), \quad q_{n_t,k} = \rho_1(\xi_k), \quad (7)$$

i.e. $D_t q = \mathbf{D}_t q_{\cdot,k} + r_t$, with $\mathbf{D}_t \in \mathbb{R}^{n_t \times (n_t-1)}$ bidiagonal and r_t the boundary contribution:

$$\mathbf{D}_t = \begin{pmatrix} 1/\Delta t_0 & & & \\ -1/\Delta t_1 & 1/\Delta t_1 & & \\ & \ddots & \ddots & \\ & & -1/\Delta t_{n_t-2} & 1/\Delta t_{n_t-2} \\ & & & -1/\Delta t_{n_t-1} \end{pmatrix}, \quad (r_t)_{i,k} = \begin{cases} -\rho_0(\xi_k)/\Delta t_0, & i = 0, \\ +\rho_1(\xi_k)/\Delta t_{n_t-1}, & i = n_t - 1, \\ 0, & \text{else.} \end{cases} \quad (8)$$

Time average $\bar{\mathbf{I}}_t$ (same domain/BCs as $\mathbf{D}_t$, used only inside the diffusion term – this is exactly Lemma 1 of `nonuniform_grid_refine_ends.tex`, the exact midpoint interpolant):

$$(\bar{I}_t q)_{i,k} = \frac{1}{2}(q_{i,k} + q_{i+1,k}) = (\bar{\mathbf{I}}_t q_{\cdot,k})_i + (s_t)_{i,k}, \quad \bar{\mathbf{I}}_t = \frac{1}{2} \begin{pmatrix} 1 & & & \\ 1 & 1 & & \\ & \ddots & \ddots & \\ & & 1 & 1 \\ & & & 1 \end{pmatrix}, \quad (9)$$

$(s_t)_{i,k} = \rho_0(\xi_k)/2$ at $i = 0$, $\rho_1(\xi_k)/2$ at $i = n_t - 1$, else 0.

Space divergence $\mathbf{D}_x$ (acts on each time row $b_{i,\cdot} \in \mathbb{R}^{n_x-1}$ independently; homogeneous – no-flux $b_{i,0} = b_{i,n_x} = 0$ needs no affine term):

$$(D_x b)_{i,k} = \frac{b_{i,k} - b_{i,k-1}}{\Delta x} = (\mathbf{D}_x b_{i,\cdot})_k, \quad \mathbf{D}_x = \frac{1}{\Delta x} \begin{pmatrix} 1 & & & \\ -1 & 1 & & \\ & \ddots & \ddots & \\ & & -1 & 1 \\ & & & -1 \end{pmatrix} \in \mathbb{R}^{n_x \times (n_x-1)}. \quad (10)$$

Space Laplacian $\mathbf{L}_x$ (acts on each time row of $\bar{I}_t q$ independently; homogeneous Neumann via ghost cells $g_0 := g_1$, $g_{n_x+1} := g_{n_x}$, so the two corner entries are -1 instead of -2):

$$(L_x[g])_{i,k} = \frac{g_{i,k+1} - 2g_{i,k} + g_{i,k-1}}{\Delta x^2} = (\mathbf{L}_x g_{i,\cdot})_k, \quad \mathbf{L}_x = \frac{1}{\Delta x^2} \begin{pmatrix} -1 & 1 & & & \\ 1 & -2 & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & 1 & -2 & 1 \\ & & & 1 & -1 \end{pmatrix} \in \mathbb{R}^{n_x \times n_x}. \quad (11)$$

8.4 Assembled

$$R(q, b) = D_t q + D_x b - \varepsilon L_x [\bar{I}_t q] \in \mathbb{R}^{n_t \times n_x}, \quad (12)$$

separable into a pure-time piece ($\mathbf{D}_t, \bar{\mathbf{I}}_t$, one independent (n_t-1) - or n_t -vector problem per space column k) and a pure-space piece ($\mathbf{D}_x, \mathbf{L}_x$, one independent (n_x-1) - or n_x -vector problem per time row i), never mixing the two within a single matrix.

$x = (q, b)$'s ambient space (1) only counts the *free* (interior) entries – $\rho_0, \rho_1, 0$ are fixed data, not unknowns – so as a map on that space, R is affine, not linear, and the boundary contribution belongs explicitly on the right-hand side, not folded silently into “ $D_t q$ ”/“ $\bar{I}_t q$ ”. Writing $\mathbf{D}_t, \bar{\mathbf{I}}_t$ acting on the free interior columns $q_{:,k}$ alone (dropping r_t, s_t from §8.3):

$$R(q, b) = \mathbf{D}_t q + \mathbf{D}_x b - \varepsilon \mathbf{L}_x [\bar{\mathbf{I}}_t q] + r, \quad r_{i,k} = \begin{cases} -\frac{\rho_0(\xi_k)}{\Delta t_0} - \varepsilon (\mathbf{L}_x [\rho_0/2])_k, & i = 0, \\ +\frac{\rho_1(\xi_k)}{\Delta t_{n_t-1}} - \varepsilon (\mathbf{L}_x [\rho_1/2])_k, & i = n_t - 1, \\ 0, & \text{else.} \end{cases} \quad (13)$$

Two contributions land in each of rows $i = 0, n_t-1$: the direct $\pm \rho_\bullet / \Delta t_\bullet$ term from $\mathbf{D}_t$'s own boundary rows (r_t), and, since $\bar{\mathbf{I}}_t q$'s boundary rows feed straight into the *linear* operator $\mathbf{L}_x$ (diffusion doesn't stop at the time boundary – $\bar{I}_t q$ at $i = 0$ is already ρ_0 -weighted, and $\mathbf{L}_x$ still acts on it in space), a second term $-\varepsilon \mathbf{L}_x [s_t]$, i.e. the spatial Laplacian of the boundary *density profile itself* ($\rho_0(\cdot)/2$, not just its value at one point) – this second piece is easy to drop (it vanishes only if ρ_0, ρ_1 happen to be spatially constant) and is exactly the part missing from (12)'s more compact notation. $D_x b$ carries no such term: b 's boundary data is the homogeneous 0, so $\mathbf{D}_x b + 0 = D_x b$ already. The FP-feasible set is

$$X = \{(q, b) : R(q, b) = 0\}, \quad (14)$$

and $f_1(x) = \iota_X(x)$ (the $\{0, +\infty\}$ indicator) – so prox_{f_1} is exactly the Euclidean projection onto X , i.e. `proj_fokker_planck_banded_nu`. *Solving* that projection (the $RR^* \phi = R(x)$ system) is a later step, not here.

To verify before Step 4

1. Is (6) the constraint you meant, with both the advective *and* diffusive flux vanishing at the space boundary?
2. Landing location: $R(q, b)$ lives on the same (τ_i, ξ_k) grid as y , not on x 's own (staggered) grid – correct?
3. Do $\mathbf{D}_t, \bar{\mathbf{I}}_t, \mathbf{D}_x, \mathbf{L}_x$ in §8.3 match what you had in mind, in particular $\mathbf{L}_x$'s -1 (not -2) corner entries from the homogeneous-Neumann ghost?
4. Does (13)'s boundary term r now match what you had in mind – both the direct $\mathbf{D}_t$ piece *and* the $-\varepsilon \mathbf{L}_x [\rho_0/2] / -\varepsilon \mathbf{L}_x [\rho_1/2]$ piece coming from the diffusion acting on the boundary-averaged density?

9 Step 4: the linearized ADMM iteration (abstract)

Deliberately abstract: no formula for A , no choice of $\langle \cdot, \cdot \rangle_{\mathcal{X}}, \langle \cdot, \cdot \rangle_{\mathcal{Y}}$ yet. The point is to pin down the *algorithm* first, see exactly where $\mathcal{X}, \mathcal{Y}$ enter it, and only then (a later step) choose them – rather than assuming Euclidean, as §7’s dropped §7.3 did, before checking whether the algorithm actually needs that choice.

9.1 Setup

$x \in \mathcal{X}$ ($= (q, b)$ -space, (1)), $y \in \mathcal{Y}$ ($= (\rho, m)$ -space), each equipped with an inner product $(\langle \cdot, \cdot \rangle_{\mathcal{X}}, \langle \cdot, \cdot \rangle_{\mathcal{Y}})$, hence norms $\| \cdot \|_{\mathcal{X}}, \| \cdot \|_{\mathcal{Y}}$ left **unspecified**. $f_1 : \mathcal{X} \rightarrow (-\infty, +\infty]$ is Step 3’s FP-feasibility indicator ι_X ; $f_2 : \mathcal{Y} \rightarrow (-\infty, +\infty]$ is Step 2’s kinetic energy (5). $A : \mathcal{X} \rightarrow \mathcal{Y}$ and $B : \mathcal{Y} \rightarrow \mathcal{Y}$ are the linear consensus maps of the constraint $Ax + By = d$ (d , not `ladmm.py`’s `b`, to stay clear of $x = (q, b)$ ’s own b – §1); this problem’s $B = -\text{Id}_{\mathcal{Y}}$ is substituted directly below (every $By \rightarrow -y$), while d is left symbolic – pinned down concretely in Step 5 (§10). A itself is not yet specified (“ $A = I_t$ ” was a placeholder name, not a claim).

9.2 The iteration

This is Fang, He, Liu & Yuan (2015)’s linearized-ADMM iteration – the one `ladmm.py` cites and implements – written with γ for their penalty parameter ρ , z^k for their dual iterate γ^k (to free up γ ; also k , not t , for the iteration index, so it doesn’t clash with the PDE time variable t used everywhere else in this note), the unlabeled norm as $\| \cdot \|_{\mathcal{Y}}$ and the G -metric as $\| \cdot \|_{\mathcal{Z}}$ (a third, still-unspecified metric – not assumed equal to $\langle \cdot, \cdot \rangle_{\mathcal{X}}$, since $\| \cdot \|_{\mathcal{Y}}$ ’s argument lives in $\mathcal{Y}$, $\| \cdot \|_{\mathcal{Z}}$ ’s in $\mathcal{X}$, but there is no reason yet to expect the same metric governs both the x -update’s proximal term and $\mathcal{X}$ itself), the transpose pairings as $\mathcal{Y}$ -inner-products ($\mathbf{x}^T \mathbf{A}^T \gamma^k = \langle Ax, z^k \rangle_{\mathcal{Y}}$, similarly for B), $B = -\text{Id}_{\mathcal{Y}}$ substituted ($By \rightarrow -y$ throughout), and over-relaxation fixed at $\alpha = 1$ (dropping every $(1 - \alpha)(\cdot)$ term):

$$x^{k+1} = \arg \min_{x \in \mathcal{X}} \left\{ f_1(x) - \langle Ax, z^k \rangle_{\mathcal{Y}} + \frac{\gamma}{2} \|Ax - y^k - d\|_{\mathcal{Y}}^2 + \frac{1}{2} \|x - x^k\|_{\mathcal{Z}}^2 \right\}, \quad (15)$$

$$y^{k+1} = \arg \min_{y \in \mathcal{Y}} \left\{ f_2(y) + \langle y, z^k \rangle_{\mathcal{Y}} + \frac{\gamma}{2} \|Ax^{k+1} - y - d\|_{\mathcal{Y}}^2 \right\}, \quad (16)$$

$$z^{k+1} = z^k - \gamma(Ax^{k+1} - y^{k+1} - d). \quad (17)$$

Everything above is well-posed as *stated* regardless of $\langle \cdot, \cdot \rangle_{\mathcal{X}}, \langle \cdot, \cdot \rangle_{\mathcal{Y}}, \langle \cdot, \cdot \rangle_{\mathcal{Z}}$ – but which point each argmin actually picks out, and hence whether (15) is `proj_fokker_planck_banded_nu` or (16) is `prox_ke_cc_nu`, depends on that choice. None of the three is assumed here.

To verify

1. Do (15)–(17) match what you had in mind – in particular the inner-product pairings $\langle Ax, z^k \rangle_{\mathcal{Y}}, \langle y, z^k \rangle_{\mathcal{Y}}$ (rather than, say, $\langle x, A^* z^k \rangle_{\mathcal{X}}$, the equivalent $\mathcal{X}$ -side pairing), and the sign flip on the y -update’s linear term from substituting $B = -\text{Id}_{\mathcal{Y}}$?
2. x, y, A, B, d as in §9.1 – anything there you’d rather state differently before we go further?
3. Next: pin down A concretely, or $\langle \cdot, \cdot \rangle_{\mathcal{X}}, \langle \cdot, \cdot \rangle_{\mathcal{Y}}, \langle \cdot, \cdot \rangle_{\mathcal{Z}}$ directly?

10 Step 5: A and d , concretely

10.1 As coded

pipeline_nu.py's consensus map:

```
def A_fn(x):
    return State(
        rho=ops.interp_t_at_phi(x.rho, rho0, rho1),
        mx=ops.interp_x_at_phi(x.mx, zeros_nt, zeros_nt),
    )
```

time-average q (boundary data ρ_0, ρ_1), space-average b (boundary data 0, the same no-flux condition §8.3 already used) – exactly “the interpolation operator in time and space (averaging)” you described, and exactly §8.3's $\bar{I}_t$ applied to q , plus a new space twin applied to b .

10.2 Splitting off the boundary part

$\bar{I}_t q$ already has its affine part identified (§8.3): $\bar{I}_t q = \bar{\mathbf{I}}_t q + s_t$. The space average of b needs its own building block, $\bar{\mathbf{I}}_x$ – same construction as $\mathbf{D}_x$ (§8.3: same domain $b_{i,\cdot} \in \mathbb{R}^{n_x-1}$, same $b_{i,0} = b_{i,n_x} = 0$ boundary), averaging instead of differencing:

$$(\bar{I}_x b)_{i,k} = \frac{1}{2}(b_{i,k-1} + b_{i,k}) = (\bar{\mathbf{I}}_x b_{i,\cdot})_k, \quad \bar{\mathbf{I}}_x = \frac{1}{2} \begin{pmatrix} 1 & & & \\ 1 & 1 & & \\ & \ddots & \ddots & \\ & & 1 & 1 \\ & & & 1 \end{pmatrix} \in \mathbb{R}^{n_x \times (n_x-1)}, \quad (18)$$

with **no** affine term: the boundary value being averaged in is 0, the same reason $\mathbf{D}_x$ needed none.

Writing $A(q, b) := (\bar{\mathbf{I}}_t q, \bar{\mathbf{I}}_x b)$ for the purely linear part (acting on the free/interior (q, b) only):

$$A_{\text{full}}(q, b) = A(q, b) + (s_t, 0). \quad (19)$$

10.3 The constant d

The original (code-level) constraint is $A_{\text{full}}(x) - y = 0$ ($B = -\text{Id}_{\mathcal{Y}}$, code's $\mathbf{b} = 0$, §1). Substituting (19):

$$A(x) + (s_t, 0) - y = 0 \iff A(x) - y = -(s_t, 0),$$

i.e., matching $Ax + By = d$ ($B = -\text{Id}_{\mathcal{Y}}$):

$$d = -(s_t, 0) \in \mathcal{Y}, \quad (d_\rho)_{i,k} = \begin{cases} -\rho_0(\xi_k)/2, & i = 0, \\ -\rho_1(\xi_k)/2, & i = n_t - 1, \\ 0, & \text{else,} \end{cases} \quad d_m = 0. \quad (20)$$

d lives only in the density slot of $\mathcal{Y}$, only in its two boundary time-rows, and is exactly minus the same boundary average s_t already computed for the FP residual in §8.3 – no new boundary data, just the other place it enters.

Step 5: confirmed

1. $A(q, b) = (\bar{\mathbf{I}}_t q, \bar{\mathbf{I}}_x b) - (19)$ minus its affine part. Confirmed.
2. (20)'s d , including the momentum slot $d_m = 0$. Confirmed.

With A and d pinned down, the open question is now exactly what §9 deferred: what $\langle \cdot, \cdot \rangle_{\mathcal{X}}, \langle \cdot, \cdot \rangle_{\mathcal{Y}}, \langle \cdot, \cdot \rangle_{\mathcal{Z}}$ should be. Per §9–§10, each is (has to be, for the algorithm in §9 to be well-posed at all) an SPD matrix norm $\|v\|_M = \sqrt{v^T M v}$; the next step is choosing $M_{\mathcal{X}}, M_{\mathcal{Y}}, M_{\mathcal{Z}}$.

11 Step 6: Pinning down $\mathcal{X}, \mathcal{Y}, \mathcal{Z}$

11.1 $\mathcal{X}$ and $\mathcal{Y}$

$\langle \cdot, \cdot \rangle_{\mathcal{X}} := \langle \cdot, \cdot \rangle_N, \langle \cdot, \cdot \rangle_{\mathcal{Y}} := \langle \cdot, \cdot \rangle_M$, where N, M are the quadrature-consistent diagonal matrices of `nonuniform_grid_norms.tex` §1 – already `pipeline_nu.py`'s `norm_fn/dual_norm_fn`, promoted here from diagnostic to the actual ADMM metric. That companion note also derives A 's N, M -weighted adjoint $A^* = N^{-1} A^T M$ (equivalently $NA^* = A^T M$, used repeatedly below) in closed form, and proves $\|A^* A\|_N = \|A\|_{\mathcal{X} \rightarrow \mathcal{Y}}^2 \leq 1$ – both facts this section builds on directly.

11.2 $\mathcal{Z}$

Define $G := \tau I - \gamma A^* A$ (an operator $\mathcal{X} \rightarrow \mathcal{X}$; $A^* A$ as a plain matrix is $N^{-1} A^T M A$, so $G = \tau I - \gamma N^{-1} A^T M A$). Take

$$\langle \cdot, \cdot \rangle_{\mathcal{Z}} := \langle \cdot, \cdot \rangle_{NG}, \quad \|v\|_{\mathcal{Z}}^2 := v^T (NG) v, \quad (21)$$

i.e. $M_{\mathcal{Z}} = NG$.

Remark 1. Order matters here: $NG \neq GN$ in general, and only NG is symmetric. $(NG)^T = G^T N = (\tau I - \gamma A^T M A N^{-1}) N = \tau N - \gamma A^T M A = NG$ (using $NA^* = A^T M \Rightarrow A^T M A N^{-1} = (NA^* A) N^{-1}$, so $G^T = \tau I - \gamma A^T M A N^{-1}$) – so NG is genuinely symmetric, hence a valid candidate for an SPD matrix norm. GN , by the same computation with the factors swapped, is symmetric only if N and $A^* A$ commute, which they have no reason to here. So $\|\cdot\|_{\mathcal{Z}}$ must be built from NG , not GN .

11.3 Collapsing the x -update to a projection under N

Substituting $\mathcal{X} = N, \mathcal{Y} = M, \mathcal{Z} = NG$ into (15), expand using $\langle Ax, z^k \rangle_M = \langle x, A^* z^k \rangle_N$ and $e^k := y^k + d$:

$$-\langle Ax, z^k \rangle_M + \frac{\gamma}{2} \|Ax - e^k\|_M^2 = \frac{\gamma}{2} \langle x, A^* A x \rangle_N - \langle x, A^* (z^k + \gamma e^k) \rangle_N + \text{const.}$$

Writing every term as a plain array pairing ($\langle x, w \rangle_N = x^T N w$, so $\langle x, A^* A x \rangle_N = x^T (N A^* A) x = x^T (A^T M A) x$) and adding $\frac{1}{2} \|x - x^k\|_{\mathcal{Z}}^2 = \frac{1}{2} x^T (NG) x - x^T (NG) x^k + \text{const.}$, the full x -dependent part of (15) is

$$x^T \left[\frac{\gamma}{2} A^T M A + \frac{1}{2} NG \right] x - x^T \left[N A^* (z^k + \gamma e^k) + N G x^k \right] + \text{const.}$$

Quadratic term: $\frac{\gamma}{2}A^TMA + \frac{1}{2}NG = \frac{\gamma}{2}NA^*A + \frac{1}{2}N(\tau I - \gamma A^*A) = \frac{\tau}{2}N$ – the A^*A pieces cancel *exactly*, which is the whole point of (21)’s choice of G : it is *forced*, not guessed, by requiring this cancellation. **Linear term:** matching $x^T[\dots]$ against $\tau x^T N w^k$ (the linear term of $\frac{\tau}{2}\|x - w^k\|_N^2$) and dividing by N (invertible):

$$\tau w^k = A^*(z^k + \gamma e^k) + Gx^k = A^*(z^k + \gamma e^k) + \tau x^k - \gamma A^*Ax^k = \tau x^k + A^*[z^k + \gamma(e^k - Ax^k)].$$

Defining $r^k := Ax^k - y^k - d$ (the constraint violation at the *current*, not yet updated, iterate – note the $-d$: with d generally nonzero (§10), this corrects the earlier abstract iteration’s implicit $d = 0$ assumption) so that $e^k - Ax^k = -r^k$, and $g^k := A^*(\gamma r^k - z^k)$:

$$w^k = x^k - \frac{1}{\tau}g^k, \quad x^{k+1} = \arg \min_{x \in \mathcal{X}} \left\{ f_1(x) + \frac{\tau}{2}\|x - w^k\|_N^2 \right\}. \quad (22)$$

This is exactly a projection (w.r.t. N) of the point $x^k - g^k/\tau$ onto the FP-feasible set $X = \{R = 0\}$ (§8.4) – the same “linearized gradient step, then prox” shape the abstract iteration always had in spirit, now derived (not assumed) from the concrete choice of $\mathcal{Z}$. Two separate pieces of machinery are involved, and only one is done: computing g^k needs A ’s weighted adjoint A^* , already in hand (`nonuniform_grid_norms.tex` §4–§5); *solving* (22)’s projection needs R ’s own weighted adjoint (`projection_nu.py`’s `_probe_composite_ops/thomas_batch_solve_general` machinery, currently assembled by probing R ’s *Euclidean* transpose) – built the same way A_t^* was, but for R ’s different combination of $\mathbf{D}_t, \bar{\mathbf{I}}_t, \mathbf{D}_x, \mathbf{L}_x$ (§8.3), plus a still-unassigned metric on the residual/dual-potential ϕ space. Not yet derived.

11.4 $G \succeq 0$ is automatic

(21) is only a valid metric if $M_{\mathcal{Z}} = NG$ is SPD, i.e. $G \succeq 0$, i.e. $\tau I \succeq \gamma A^*A$, i.e. $\tau \geq \gamma \|A\|_{\mathcal{X} \rightarrow \mathcal{Y}}^2$ (the generalized-Rayleigh-quotient characterization of the operator norm). This is exactly `nonuniform_grid_norms.tex` §7’s stability condition – already proved to hold for $\tau > \gamma$, unconditionally, on any grid in this family. So $\mathcal{Z}$ is guaranteed well-defined by a bound already established, not a new condition to check.

11.5 The iteration, purely in terms of N and M

Collecting: (15)–(17) with every $\mathcal{X}, \mathcal{Y}, \mathcal{Z}$ replaced by N, M, NG and simplified as far as it goes.

x -update – already done, (22):

$$x^{k+1} = \arg \min_{x \in \mathcal{X}} \left\{ f_1(x) + \frac{\tau}{2}\|x - w^k\|_N^2 \right\}, \quad w^k = x^k - \frac{1}{\tau}A^*(\gamma r^k - z^k), \quad r^k = Ax^k - y^k - d. \quad (23)$$

No further simplification is available here – $f_1 = \iota_X$ isn’t itself a sum of independent per-point pieces (§8: R couples neighboring t_j ’s and x_k ’s), so (23) is already in its simplest form; *solving* it is §11.3’s open item, not a simplification of the argmin’s statement.

y -update – same completing-the-square move as (22), applied to (16) with $\mathcal{Y} = M$ ($v^k := (Ax^{k+1} - d) - z^k/\gamma$):

$$y^{k+1} = \arg \min_{y \in \mathcal{Y}} \left\{ f_2(y) + \frac{\gamma}{2}\|y - v^k\|_M^2 \right\}.$$

Unlike f_1 , f_2 is a sum of independent per-point terms ((5): $f_2(y) = \sum_{i,k} \Delta t_i \Delta x L(\rho_{i,k}, m_{i,k})$), and M carries *the same* per-point weight $\Delta t_i \Delta x$ by construction (§11.1) – so $\|y - v^k\|_M^2 =$

$\sum_{i,k} \Delta t_i \Delta x [(\rho_{i,k} - v_{\rho,i,k}^k)^2 + (m_{i,k} - v_{m,i,k}^k)^2]$ shares the identical weight, term by term, with f_2 . The full objective is then $\sum_{i,k} \Delta t_i \Delta x [L(\cdot, \cdot) + \frac{\gamma}{2}(\cdot, \cdot)]$, a sum of independent terms each scaled by a positive constant that cannot move that term's own minimizer – so the $\Delta t_i \Delta x$ cancels entirely, point by point:

$$(\rho_{i,k}^{k+1}, m_{i,k}^{k+1}) = \arg \min_{\rho, m} \left\{ L(\rho, m) + \frac{\gamma}{2} [(\rho - v_{\rho,i,k}^k)^2 + (m - v_{m,i,k}^k)^2] \right\}, \quad \forall (i, k). \quad (24)$$

No Δt_i , no Δx , anywhere – (24) is *exactly* `prox_ke_cc`'s uniform-grid cubic solve, at the constant step $\sigma = 1/\gamma$, called pointwise. This is the simplest the y -update can possibly get: strictly simpler than `prox_ke_cc_nu`'s current per-row $\sigma_{\text{eff}} = \sigma \Delta t_i / \Delta t_{\text{ref}}$ rescaling, which becomes not just unnecessary but wrong to keep once M is the genuine metric (it was compensating for a mismatch – weighted f_2 , unweighted penalty – that (24) shows doesn't exist here).

z-update – (17) is a plain vector formula, never inside a norm or inner product on either side; substituting $\mathcal{Y} = M$ changes nothing about it:

$$z^{k+1} = z^k - \gamma (Ax^{k+1} - y^{k+1} - d). \quad (25)$$

(z^k lives in $(\mathcal{Y}, M)$ – its *norm*, if ever reported as a diagnostic, would use M – but the update rule itself is metric-independent, so there is nothing left to simplify.)

To verify

1. $\mathcal{X} = N, \mathcal{Y} = M$ (§11.1) – the intended pinning?
2. $\mathcal{Z} = NG$, not GN (§11.2's Remark) – does the asymmetry argument hold up, or was GN actually intended in some other sense (e.g. as a right-acting operator, not a symmetric bilinear form)?
3. (22)'s derivation – every step, in particular the corrected $r^k := Ax^k - y^k - d$ (now including $-d$) – any gap?
4. §11.5's three final forms – (23) (unchanged/unsimplifiable), (24) (the $\Delta t_i \Delta x$ cancellation, landing exactly on `prox_ke_cc`), (25) (no simplification applies) – correct, and is (24)'s claim that `prox_ke_cc_nu`'s σ_{eff} rescaling should go away entirely (not just simplify) the conclusion you expected?
5. Next: derive R 's N -weighted adjoint (redoing §8's projection under N), which is what (23) still needs before it's fully concrete?